

USACH

# SISTEMA DE ADMINISTRACIÓN ACADÉMICA UNIVERSITARIA

Start

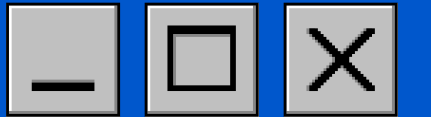
PROFESOR:  
MATÍAS CALDERÓN



AYUDANTE:  
FERNANDO SOLÍS



# Integrantes



**Inicio** Desarrollo Final

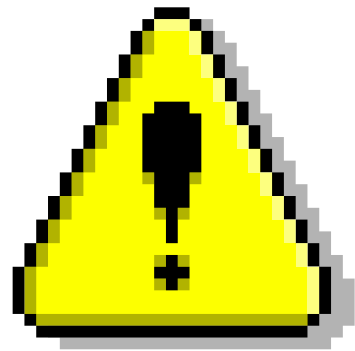
 I1: VICENTE  
TAPIA

 I2: DANIEL  
DÍAZ

 I3: BENJAMÍN  
ARAYA

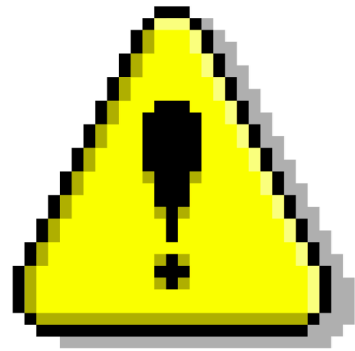
 I4: JORDAN  
GONZÁLEZ





Una universidad requiere una intranet para **centralizar** y **automatizar** la gestión académica de sus **estudiantes**, reemplazando procesos manuales o dispersos. El sistema debe manejar información de **estudiantes**, **asignaturas**, **secciones**, **semestres** y **calificaciones**, garantizando que cada usuario como **profesores**, **estudiantes** y **administradores**, acceda únicamente a la información y funciones que le corresponden según su rol.





Para esta segunda etapa, la universidad además necesita incorporar el componente **geoespacial** del campus: la ubicación real de edificios y salas, de modo que el sistema pueda orientar a los estudiantes en **tiempo real** —indicando dónde tienen clase según su posición GPS—, identificar qué salas son accesibles para personas con movilidad reducida, sugerir secciones cercanas al momento de inscribirse, y analizar el rendimiento académico también desde una **perspectiva territorial**, cruzando resultados con la zona de residencia de los estudiantes.



## ¿Qué debemos desarrollar?



Inicio Desarrollo Final

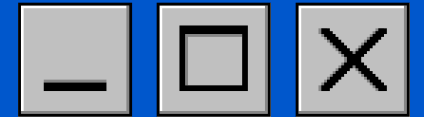


- CRUD de edificios y salas con geometrías PostGIS.
- Endpoint PostGIS: ubicación GPS del estudiante → sala más cercana con clase activa.
- Accesibilidad: salas cercanas a rampas registradas (ST\_DWithin).
- Optimización de traslados: secciones a menos de 300m del estudiante, ordenadas por distancia.
- Vista materializada: densidad estudiantil por edificio (mapa de calor).
- Vista materializada: reprobación por distrito de vivienda (ST\_Contains).
- Índices GIST en todas las columnas geométricas.
- GeoJSON vía ST\_AsGeoJSON() – sin librerías de geometría en Java (regla "sin ORM").





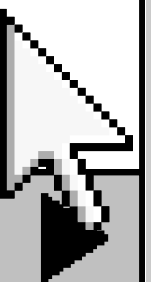
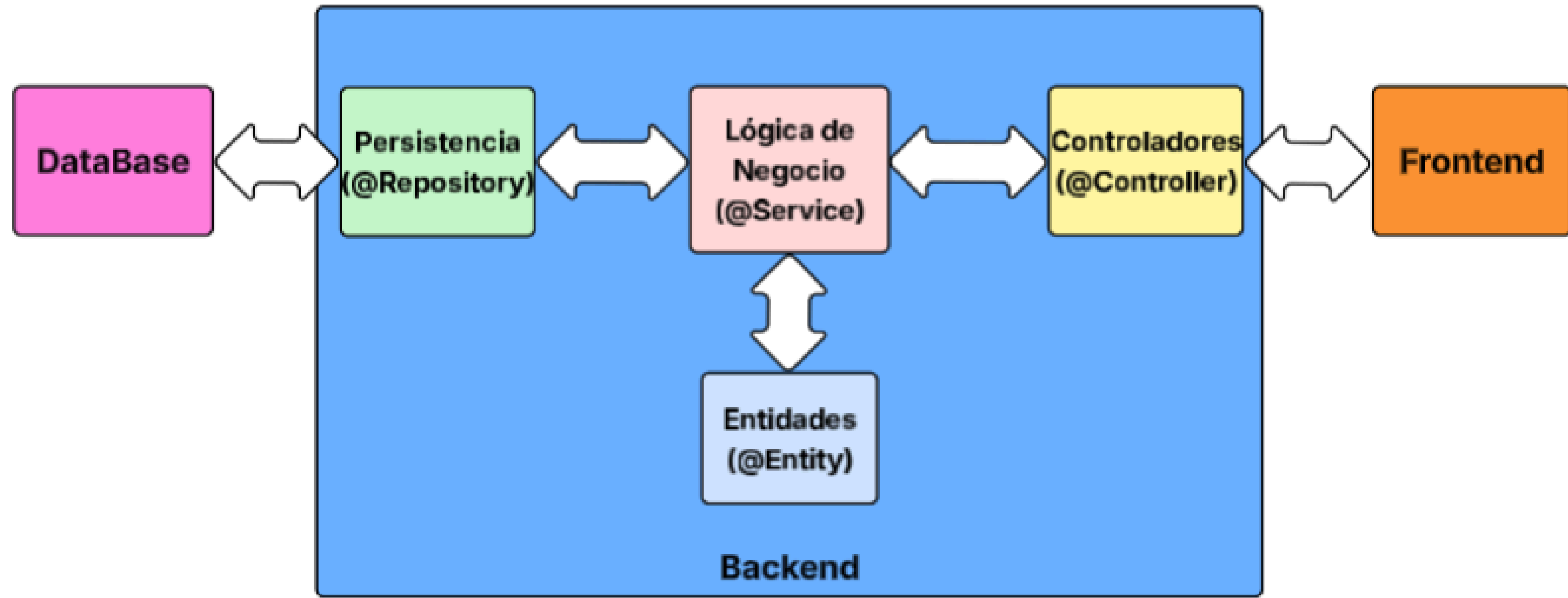
# Arquitectura



Inicio Desarrollo Final



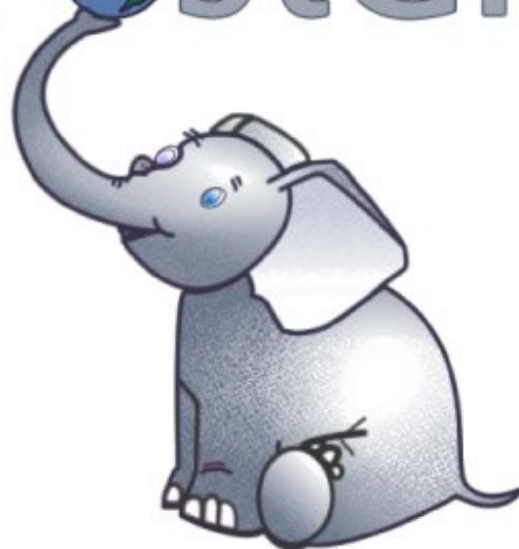
ARQUITECTURA POR  
CAPAS – MONOLÍTICA





## DATABASE

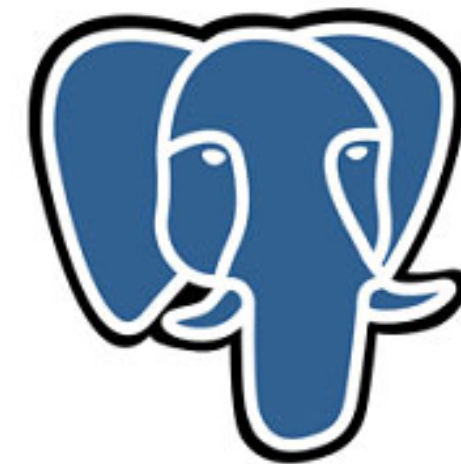
PostGIS



**GEOJSON**



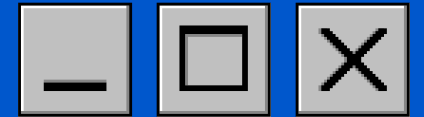
**React Leaflet**



PostgreSQL



# Tecnologías Usadas



Inicio **Desarrollo** Final



## BACKEND



v4.0.6



Dependencias usadas:

- spring-boot-starter-web
- spring-boot-starter-data-jdbc
- spring-boot-starter-security
- postgresql
- lombok
- springdoc-openapi (Swagger UI)
- spring-boot-starter-test
- jjwt-api / jjwt-impl / jjwt-jackson



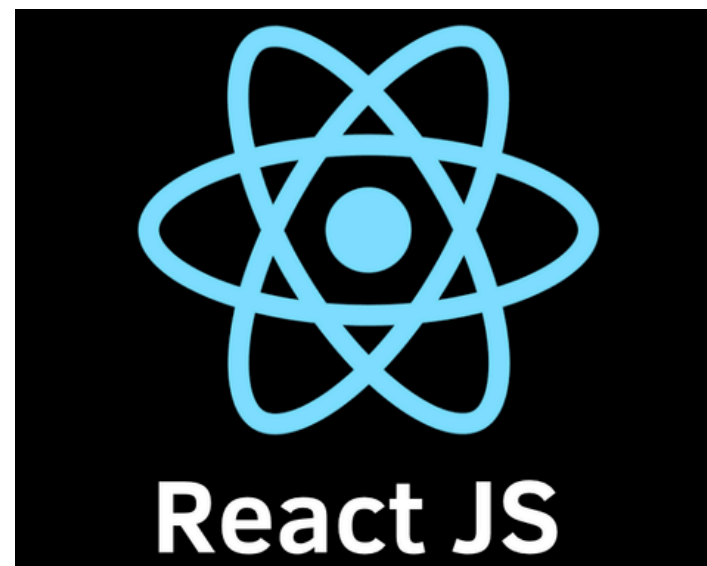
21 jdk







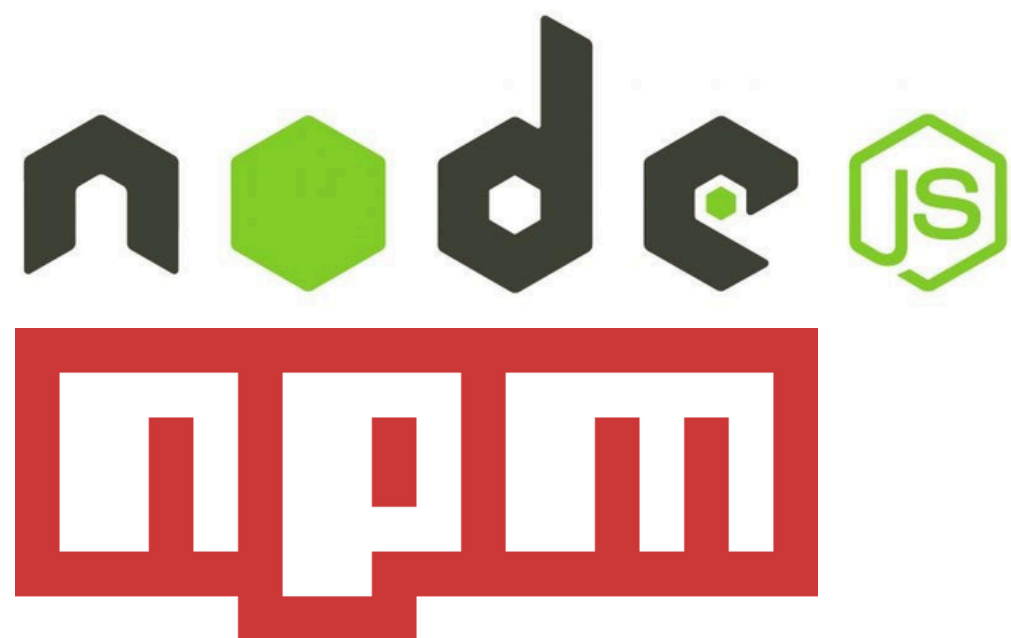
Inicio **Desarrollo** Final



◀◀◀  
v18.x



## FRONTEND



◀◀◀  
v22.22.0

◀◀◀  
v10.9.4

Librerías utilizadas en frontend:

- Bootstrap (estilo y diseño)
- Axios (HTTP → backend)
- React Router DOM (componentes)



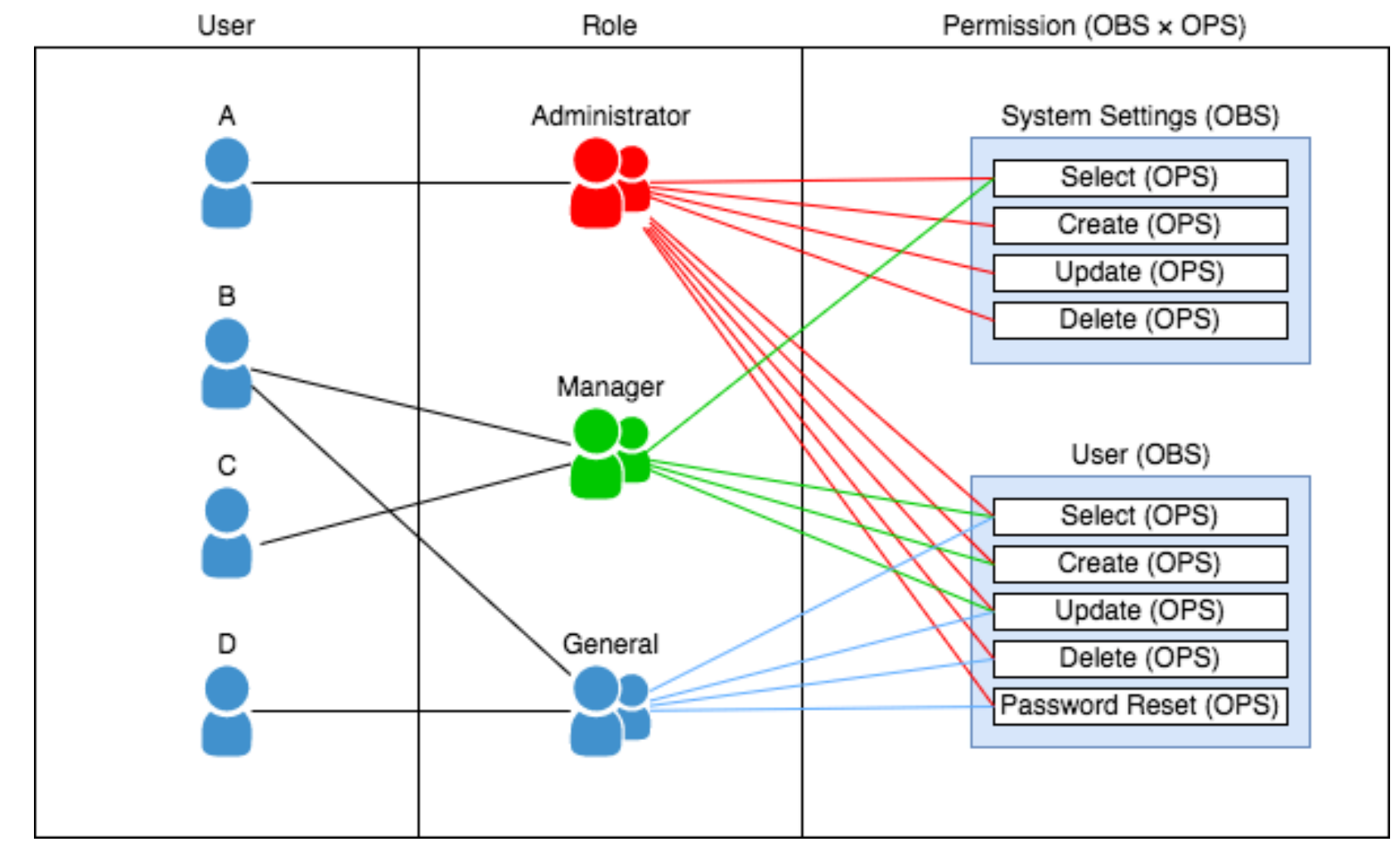


## SEGURIDAD

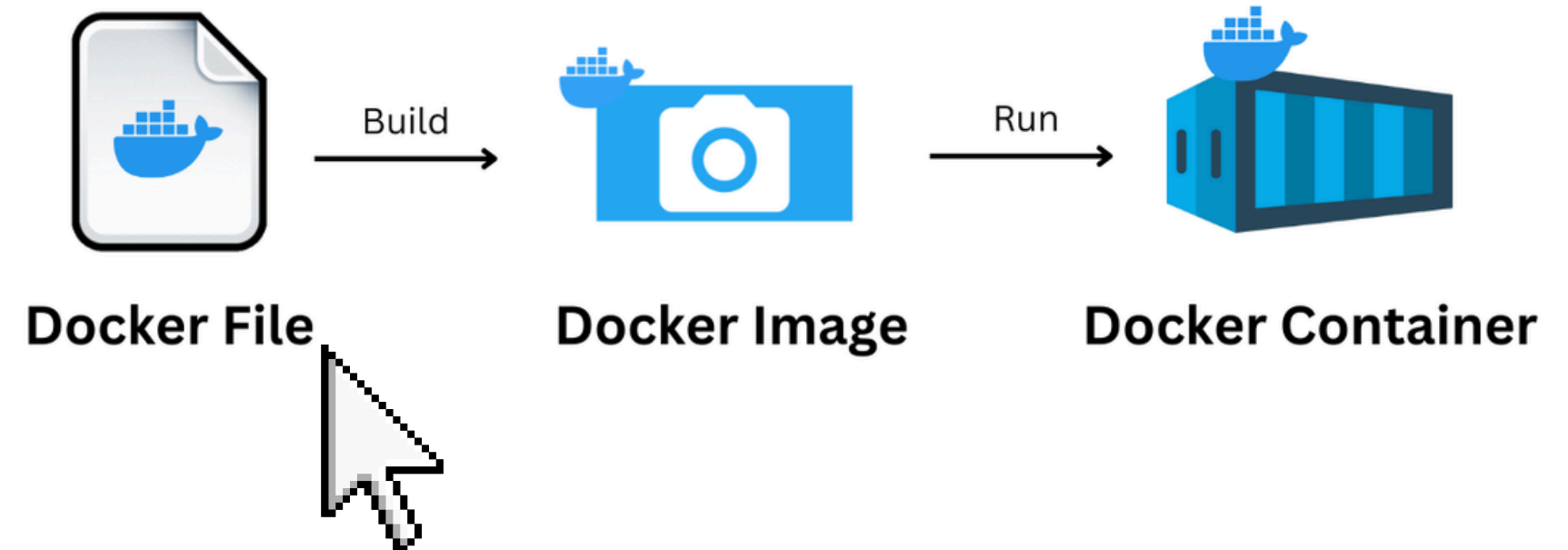
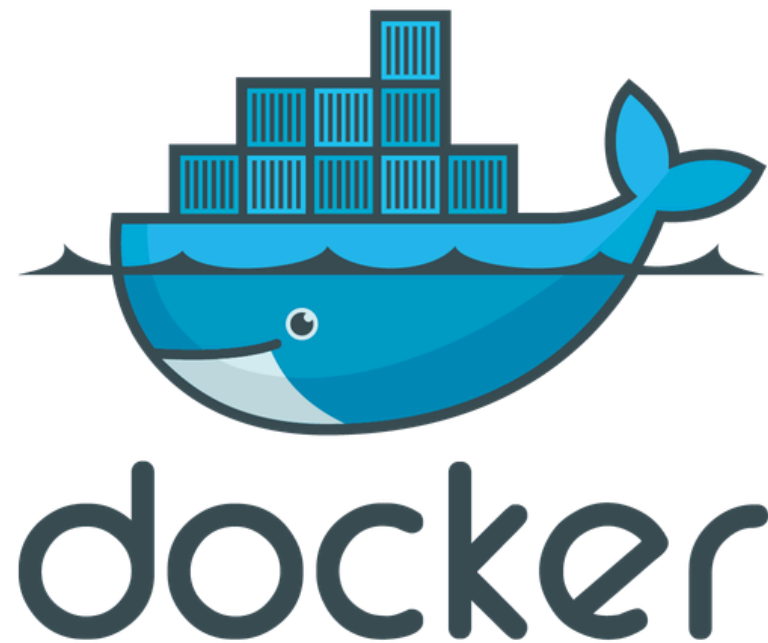
json web token

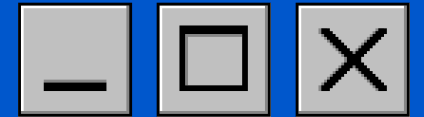


### Role-Base Access Control (RBAC)

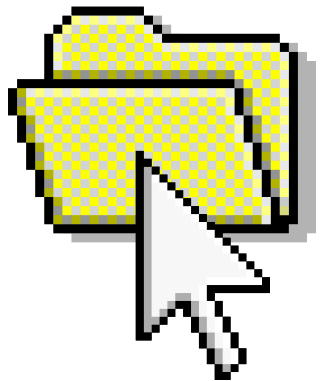


Inicio **Desarrollo** Final





Inicio Desarrollo Final



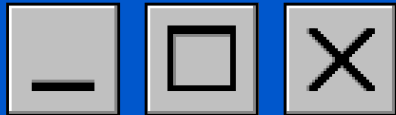
| Active branches           |  |              |              |        |       |              |
|---------------------------|--|--------------|--------------|--------|-------|--------------|
| Branch                    |  | Updated      | Check status | Behind | Ahead | Pull request |
| feature/modulo-usuario    |  | 16 hours ago |              | 6      | 23    | #4  ...      |
| feature/modulo-estudiante |  | 18 hours ago |              | 6      | 8     | #3  ...      |
| feature/modulo-academico  |  | yesterday    |              | 6      | 7     | ...          |
| feature/modulo-profesor   |  | 2 days ago   |              | 6      | 10    | #2  ...      |



**TRABAJO REALIZADO  
POR RAMAS.**



# Trabajo en equipo



Inicio    **Desarrollo**    Final



Switch branches/tags

Find or create a branch...

Branches

Tags

main

feature/i1-fundacion-espacial

feature/i2-ubicacion-accesibilidad

feature/i3-optimizacion-traslados

feature/i4-reportes-geoespaciales

feature/modulo-academico

feature/modulo-estudiante

feature/modulo-profesor

feature/modulo-usuario

lab2

View all branches

| Branch                             | Updated        | Check status | Behind | Ahead | Pull request |
|------------------------------------|----------------|--------------|--------|-------|--------------|
| feature/i2-ubicacion-accesibilidad | 23 minutes ago |              | 2      | 23    |              |
| lab2                               | 31 minutes ago |              | 2      | 34    |              |
| feature/i3-optimizacion-traslados  | 2 hours ago    |              | 2      | 31    | #9           |
| feature/i4-reportes-geoespaciales  | 18 hours ago   |              | 2      | 14    | #8           |
| feature/i1-fundacion-espacial      | 3 days ago     |              | 2      | 10    | #6           |





# Trabajo en equipo



Inicio

Desarrollo Final

## Integrante 1 – Fundación Espacial

Tablas BD: building y room

Archivos Spring Boot:

- BuildingEntity.java, RoomEntity.java, BuildingRepository.java, RoomRepository.java, BuildingService.java, RoomService.java, BuildingController.java, RoomController.java. GeoJsonUtils.java

• Endpoint clave:

- GET/POST/PUT/DELETE /api/buildings
- GET/POST/PUT/DELETE /api/rooms

Prioridad: Primero, todo depende de esto.



**DIVISIÓN PARA  
DATABASE Y BACKEND**

## Integrante 2 – Ubicación en tiempo real + Accesibilidad

Tabla BD: accessibility\_poi

Archivos Spring Boot:

LocationController.java,  
RoomAccessibilityController.java,  
AccessibilityPoiController.java, LocationService.java,  
AccessibilityPoiService.java, LocationRepository.java,  
AccessibilityPoiRepository.java

Objetos BD avanzados: Índice GIST idx\_ramp\_geom,  
consulta KNN con operador <->, ST\_Distance sobre  
geography

Endpoints clave: POST /api/location/nearest-room GET  
/api/rooms/accessible?buildingId=  
/api/accessibility-pois (solo ADMIN) CRUD



# Trabajo en equipo



Inicio    Desarrollo    Final

## Integrante 3 – Optimización de traslados

Tablas BD: No se crean tablas nuevas

### Archivos Spring Boot:

- Endpoint dentro del controller de enrollments existente (o nuevo NearbySectionsController.java)
- EnrollmentService.java – extiende lógica de consulta

### Endpoints clave:

GET /api/enrollments/nearby-sections?  
subjectId=2&lat=-33.4489&lng=-70.6693

## Integrante 4 – Vistas materializadas + Reportes

Tablas BD: housing\_district, Modificación a student

### • Archivos Spring Boot:

ReportController.java

### • Objetos BD avanzados:

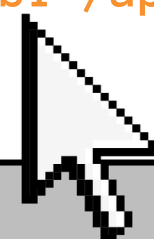
- Índices GIST: idx\_district\_geom, idx\_student\_home\_geom
- Vista materializada 1 – mv\_student\_density\_by\_building: densidad estudiantil por edificio
- Vista materializada 2 – mv\_failure\_rate\_by\_district: reprobación por distrito

### Endpoints clave:

GET /api/reports/density-heatmap → GeoJSON  
FeatureCollection

GET /api/reports/failure-by-district → GeoJSON  
FeatureCollection

POST /api/reports/refresh → refresca las vistas





# Trabajo en equipo



Inicio

Desarrollo      Final



## DIVISIÓN PARA EL FRONTEND

Integrante 1 – Fundación Espacial

Vistas: `BuildingList.jsx`,  
`BuildingForm.jsx`, `RoomList.jsx`,  
`RoomForm.jsx`

Responsabilidades extra:

Instalar y configurar react-leaflet

Construir componente base `<MapView />`  
reutilizable

Dejar `<MapView />` disponible para que los otros 3  
integrantes lo usen en sus propias vistas

Endpoint que consume: GET/POST/PUT/DELETE  
`/api/buildings`, `/api/rooms`

Prioridad: Primero – todo depende de esto.

Integrante 2 – Ubicación + Accesibilidad  
(Frontend)

Vistas: `MyLocation.jsx`, `AccessibilityBadge.jsx`

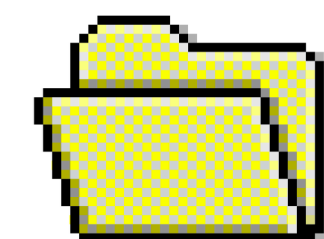
Hooks: `useGeolocation.js`

Integración en vistas de I1: badge de  
accesibilidad agregado en `RoomList.jsx` (tabla de  
Salas)

Endpoints que consume: POST

`/api/location/nearest-room` GET

`/api/rooms/accessible`





# Trabajo en equipo



Inicio

Desarrollo Final

Integrante 3 – Optimización de traslados

Vistas: EnrollForm.jsx

Hooks: useGeolocation.js

Endpoints que consume: GET

/api/enrollments/nearby-sections?

subjectId=&lat=&lng=

Integrante 4 – Vistas materializadas +  
Reportes

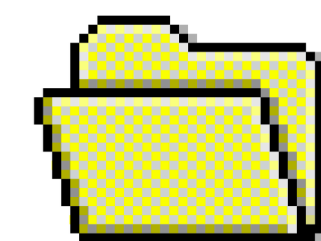
Vistas: DensityHeatmap.jsx,

DistrictFailureMap.jsx

Endpoints que consume:

GET /api/reports/density-heatmap

GET /api/reports/failure-by-  
district

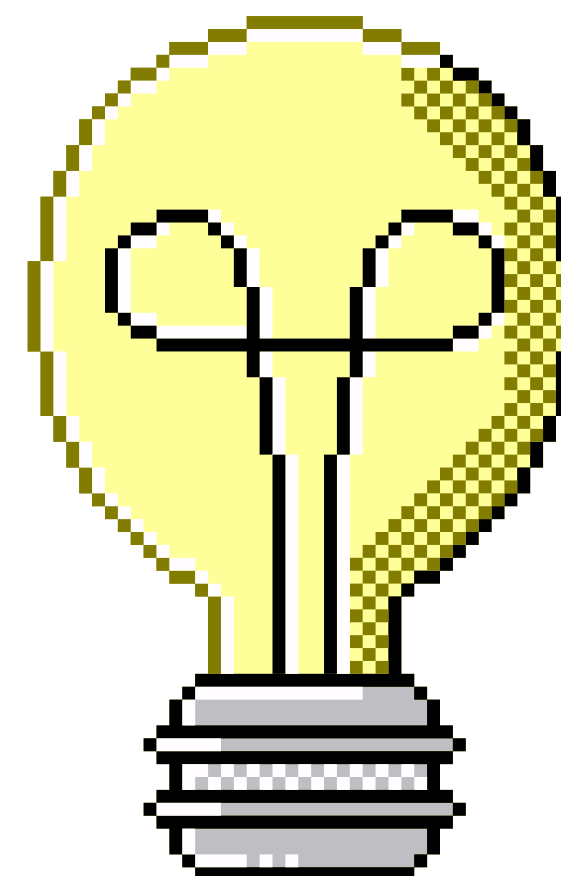


**Demostración**

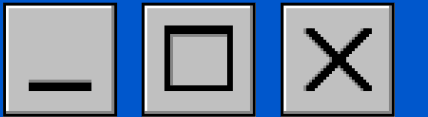


Inicio **Desarrollo** Final

**HORA DE BOOTEAR EL  
PROYECTO**



Final



Inicio Desarrollo **Final**

CONCLUSIÓN

